Assessement of COA Prepared by GROUP 4



SCHOOL OF COMPUTING AND INFORMATICS.

DEPARTEMENT OF INFORMATION SYSTEMS.

***COURSE:* COMPUTER ORGANIZATION AND ARCHITECTURE.**

Course Code :InSy2054

GROUP ASSESSMENT OF COA

Section : 'A' ***Group 4***

GROUP MEMBERS

No.	Name	Id
1.	Adugnaw Gete	MTUUR/8693/17
2.	Abyalew Tadese	MTUUR/8648/17
3.	Birtukan Bekele	MTUUR/8329/17
4.	Abaynesh Wassie	MTUUR/8551/17
5.	Daryelesh Tashome	MTUUR/8617/17
6.	Betremariam Gizachew	MTUUR/7775/17
7.	Amanuel Mosisa	MTUUR/7918/17
8.	Elias Semahegn	MTUUR/8011/17
9.	Abenezer Zewdie	MTUUR/8458/17
10.	Betel Abera	MTUUR/7733/17

MTU,Ethiopia **Submitted to: Ms. TSIGE**

Submission date: 16/08/2018E.C

Assessement of COA Prepared by GROUP 4

Table Content

Introduction ……....1

Functional organization ……...2

1. **Implementation of Simple Data Paths……...3**

1.1 Simple data path ……..3

2 .Control Unit Realization……..8

2.1, Hardwired Control ……....8

2.2, Microprogrammed Control……10

3. Instruction Pipelining ……... 12

4. **Instruction-Level Parallelism (ILP)……. 14**

Conclusion……...19

References……20

Assessement of COA Prepared by GROUP 4

Introduction

In the modern era of computing, the efficiency of a system is not merely defined by its raw speed, but by the elegance of its internal organization. **Computer Architecture** serves as the fundamental blueprint, encompassing the rules and methods that describe the functionality and implementation of computer systems. At its core, architecture balances a delicate triad of objectives: maximizing performance, minimizing power consumption, and maintaining high reliability.

The concept of **Functional Organization** is central to this design. By applying the principle of specialization, a computer system can decentralize decision-making. Specific units are delegated distinct roles such as arithmetic processing, memory management, or signal control allowing for a modular approach where each component is optimized for its specific goal. This specialization is most visible in the **Instruction Set Architecture (ISA)**, which acts as the critical interface between software and hardware. Since processors only understand binary numerical encodings, the ISA provides the "programmer's view," defining the data types, registers, and memory addressing modes that software tools like compilers use to translate high-level languages into machine-executable actions.

Beyond the abstract rules of the ISA lies **Implementation**, often categorized into logic and circuit design. While the ISA defines *what* a computer can do, the **Datapath** and **Control Unit** define *how* it is done. The datapath acts as the "brawn" of the processor, containing the Arithmetic Logic Unit (ALU), registers (such as the Program Counter and Instruction Register), and buses that facilitate the flow of data. Conversely, the Control Unit acts as the "brain," orchestrating these components to execute the fetch-decode-execute cycle.

As performance demands have evolved, so too have the strategies for handling data. Designers must choose between **Single-Cycle**, **Multi-Cycle**, or **Pipelined** datapaths. While single-cycle designs offer simplicity, modern high-performance computing relies heavily on pipelining and **Instruction-Level Parallelism (ILP)**. These techniques treat the processor like an industrial assembly line, overlapping the execution of multiple instructions to ensure that no part of the hardware sits idle. By breaking the "Power Wall" where increasing clock speeds becomes thermally unsustainable ILP has become the primary driver of performance in contemporary microprocessor design.

Assessement of COA Prepared by GROUP 4

Functional organization

- Functional organization is a type of organizational structure that uses the principle of specialization based on function or role.
- It allows decisions to be decentralized since issues are delegated to specialized persons or units, leaving them the responsibility of implementing, evaluating, or controlling the given procedures or goals.
- In computer engineering, computer architecture is a set of rules and methods that describe the functionality, organization, and implementation of computer systems.
- In other definitions computer architecture involves instruction set architecture design, micro architecture design, logic design, and implementation.
- The purpose is to design a computer that maximizes performance while keeping power consumption in check, costs low relative to the amount of expected performance, and is also very reliable.
- The implementation involves integrated circuit design, packaging, power, and cooling.
- Optimization of the design requires familiarity with compilers, operating systems to logic design, and packaging.
- An instruction set architecture (ISA) is the interface between the computer's software and hardware and also can be viewed as the programmer's view of the machine.
- Computers do not understand high-level programming language such as Java, C++, or most programming languages used.
- A processor only understands instructions encoded in some numerical fashion, usually as binary numbers.
- Software tools, such as compilers, translate those high-level languages into instructions that the processor can understand.
- Besides instructions, the ISA defines items in the computer that are available to a program.

Examples : data types, registers, addressing modes, and memory. Instructions locate these available items with register indexes (or names) and memory addressing modes.

Assessement of COA Prepared by GROUP 4

1. Implementation of Simple Data Paths

A Data path is a critical component within a processor that handles data processing operations. It works in conjunction with a control unit to form the CPU (central processing unit) of a computer system.

▪ Data path –is the collection of state elements, computation elements, and interconnections that together provide a conduit for the flow and transformation of data in the processor during execution.

1.1 Simple data path.

- Simple data path components include memory (stores the current instruction), PC or program counter (stores the address of current instruction), and ALU (executes current instruction).
- The data path is the "brawn" of a processor, since it implements the fetch-decode-execute cycle.
- The general discipline for data path design is considered:
- To determine the instruction classes and formats in the ISA,
  To design data path components and interconnections for each instruction class or format, and
- To compose the data path segments designed in Step 2 to yield a composite data path.

➢ Components of a Simple Data path:

A. ALU (Arithmetic Logic Unit): The ALU executes the current instruction and perform arithmetic and logic operation.

▪ ALU modern CPU contains a control unit and a set of registers.

▪ Most of the operations are performed by one or more ALU's, which load data from the input register.

B. Register : are like gates here through which signals are further sent to components to do the micro-operation.

Here are five registers used to store in-out signal data:

- ✓ **Program Counter**

The program counter (PC) is a CPU register in the computer processor which has the address of the next instruction to be executed from memory.

- ✓ **Instruction Register**

In computing, an instruction register (IR) is the part of a CPU's register that holds the instruction currently being executed or decoded.

- ✓ **Memory Address Register**

The Memory Address Register (MAR) is the CPU register that either store the memory address

from which data will fetch from the CPU or the address to which data will be sent and stored.

- ✓ **Memory Data Register**

The memory data register (MDR) is the register in a computer's processor, or

central processing unit (CPU) that stores the data being transferred to and from the immediate access storage.

- ✓ **General purpose register:**

General-purpose registers are used to store temporary data within the microprocessor, and it is a multipurpose register.

C. Bus : is parallel electrical wires with multiple hardware connections. Therefore a bus is a communication system that transfers data between components inside a computer or between computers.

When we would have different kinds of buses, different types of bus organizations will take place, such as:

❖ **One Bus organization**

In one bus organization, a single bus is used for multiple purposes. A set of general-purpose registers, program counters, instruction registers, memory address

registers (MAR), and memory data registers (MDR) are connected with the single bus.

❖**Two Bus organization**

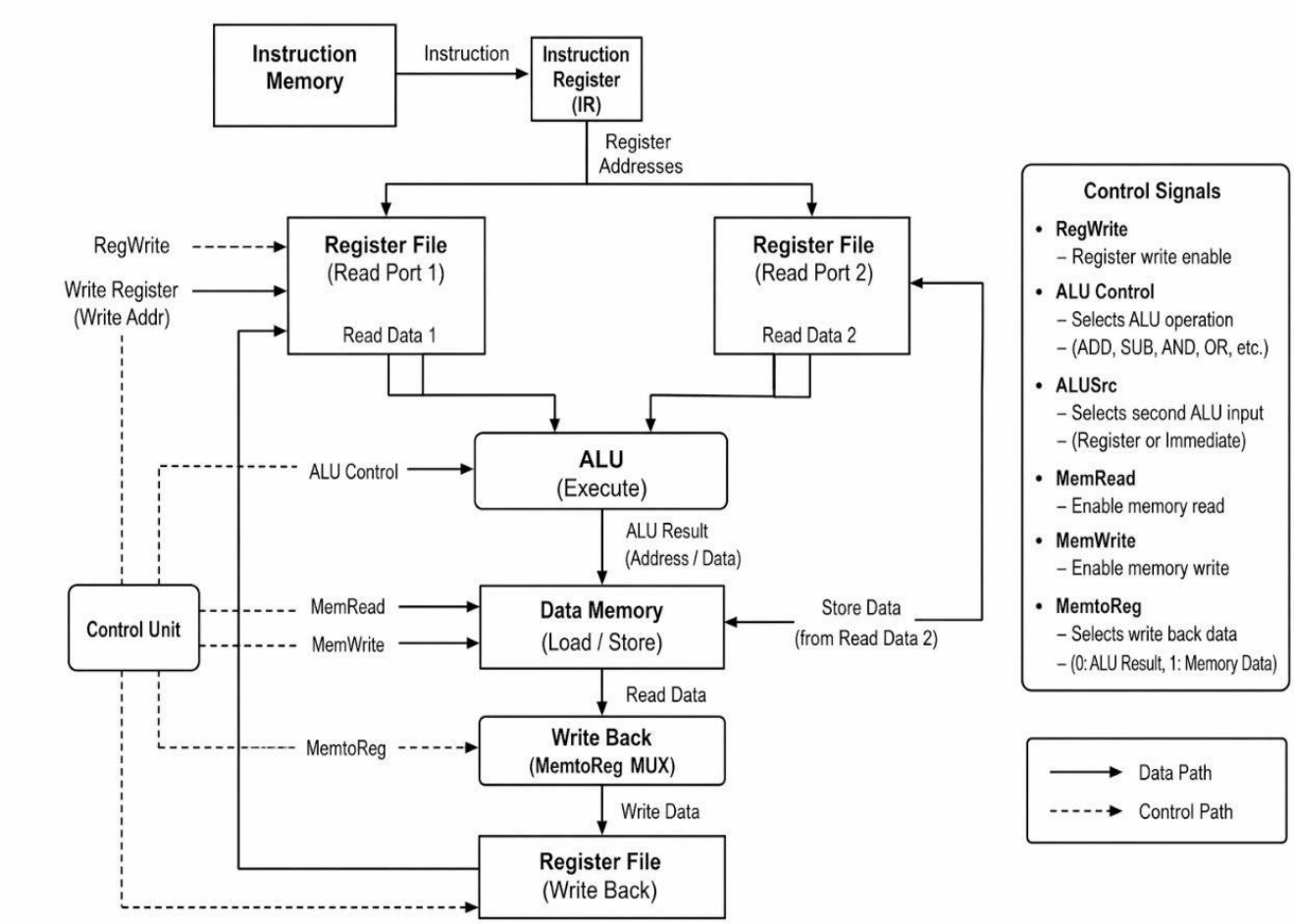
In two bus organizations, there are two buses, and the general-purpose register can read/write from both the buses. In this case, two operands can be fetched at the same time because of the two buses.

❖**Three Bus organization**

In three bus organizations, we have three buses, OUT bus1, OUT bus2, and an IN bus. From the out buses, we can get the operand that can come from the general-purpose register and evaluated in ALU, and the output is dropped on In Bus to be sent to respective registers.

D. Memory: is used to store instructions and data that the processor needs to access. It can be divided into different types, such as main memory(RAM) and secondary storage (Hard disk, SSD).

E. ControlUnit: is responsible for managing the overall operation of the data path. Control unit generates all the signals (in-out) based on which all other component works.



- **Common Data Taypes**

In computer architecture, the **datapath** is the collection of hardware functional units (such as ALUs, registers, and buses) that perform data processing operations. Datapaths are typically categorized by how they handle instructions and timing.

1. Single-Cycle Datapath

In a single-cycle design, the engine processes one entire instruction within a single clock cycle.

- **How it works:** Every instruction—from fetching it from memory to writing the result back to a register—must be completed before the next clock edge.
- **Pros:** It is conceptually simple and easy to design.

- **Cons:** The clock cycle must be long enough to accommodate the **slowest possible instruction** (usually a Load from memory). This means faster instructions (like an Add) waste time waiting for the cycle to end.
- **Efficiency:** High "Cycles Per Instruction" (CPI = 1), but low clock frequency.

2. Multi-Cycle Datapath

A multi-cycle datapath breaks an instruction down into smaller steps (e.g., Fetch, Decode, Execute, Memory Access, Write Back), with each step taking one clock cycle.

- **How it works:** Functional units can be shared. For example, a single ALU can be used for both incrementing the Program Counter and performing arithmetic, because those actions happen in different cycles.
- **Pros:** Allows for a faster clock speed because each cycle only does a small amount of work. It also saves hardware space through unit reuse.
- **Cons:** Requires more complex control logic (often a Finite State Machine) to track which step the instruction is currently in.
- **Efficiency:** Variable CPI (e.g., an Add might take 3 cycles, while a Load).

3. Pipelined Datapath

Pipelining is the most common approach in modern processors. It overlaps the execution of multiple instructions simultaneously, similar to an assembly line.

- **How it works:** While one instruction is being executed, the next one is being decoded, and the one after that is being fetched.
- **Key Components: Pipeline Registers** are placed between stages (IF, ID, EX, MEM, WB) to hold the data as it moves through the "line" on each clock pulse.
- **Pros:** Significantly increases **throughput** (the number of instructions completed per second).
- **Cons:** Introduces "hazards" (structural, data, and control) where instructions might depend on the results of others not yet finished, requiring stalls or forwarding logic.

Feature	Single-Cycle	Multi-Cycle	Pipelined
Clock Speed	Slow (Limited by longest path)	Fast	Very Fast
CPI	Exactly 1	> 1 (Variable)	Ideally 1 (due to overlap)
Hardware Usage	Redundant units (e.g., multiple adders)	Efficient (Shared units)	Complex (Pipeline registers + Hazard logic)
Best Use Case	Simple/Educational CPUs	Low-power/Embedded systems	High-performance computing

2 .Control Unit Realization

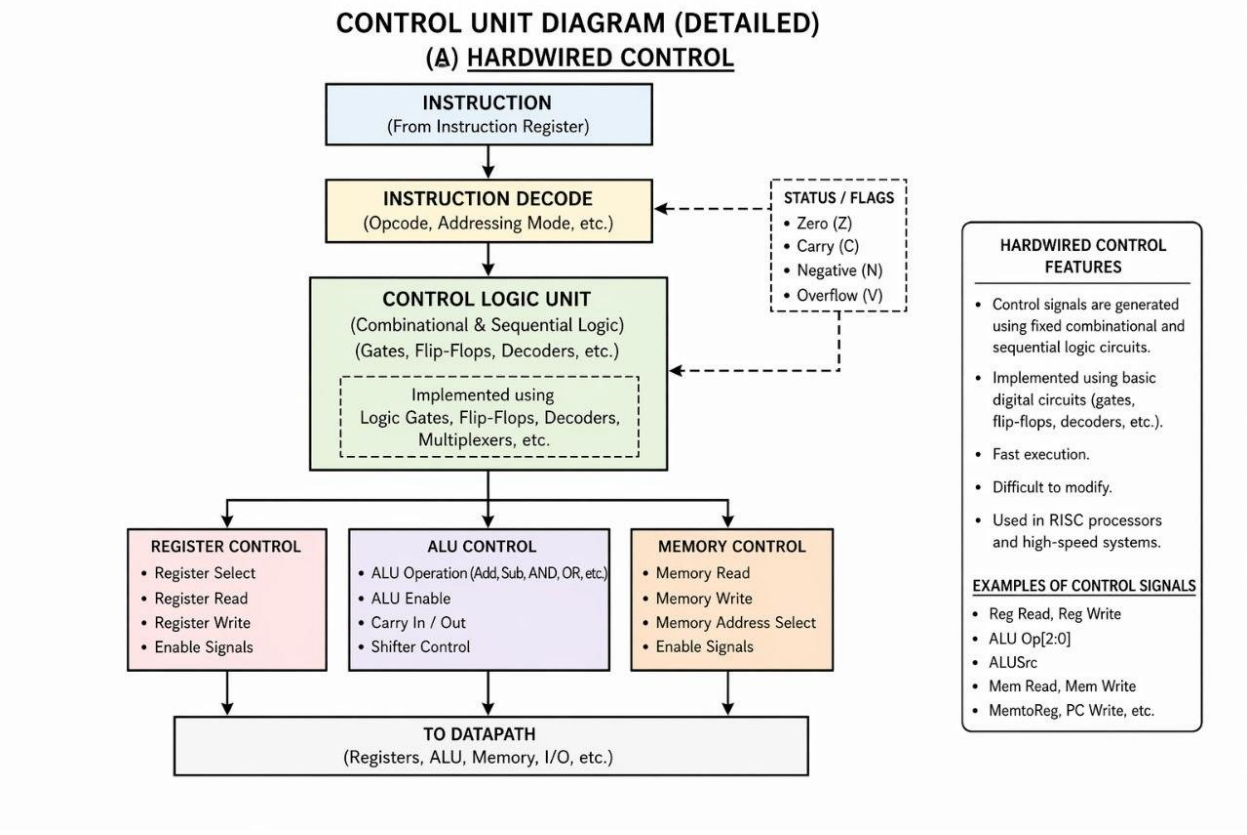
- The Control Unit (CU) is the "brain" of the processor. It directs the flow of data by generating control signals that tell the datapath components (ALU, registers, memory) exactly what to do for each instruction.
- There are two fundamental ways to design or "realize" a control unit. The choice between them depends on the complexity of the instruction set, the need for flexibility, and the target speed.

2.1 Hardwired Control

Hardwired control implements the control unit as a large, fixed collection of combinational logic gates, decoders, and flip-flops.

- **How it works:** The control signals are generated directly from the instruction opcode and the status flags using Boolean logic. The design is essentially a hardware circuit specifically tailored to the instruction set architecture (ISA).

- **Pros:**
  - **Extremely Fast:** Since it uses pure hardware logic, there is no "lookup" time; signals are generated almost instantly.
  - **Efficient:** It requires minimal overhead for simple operations.
- **Cons:**
  - **Inflexible:** If you need to update the instruction set, you must physically redesign the circuit board/chip.
  - **Complexity:** As the number of instructions increases, the logic gates required grow exponentially, making it difficult to manage.
- **Best for:** Reduced Instruction Set Computer (**RISC**) architectures where instructions are simple and uniform.

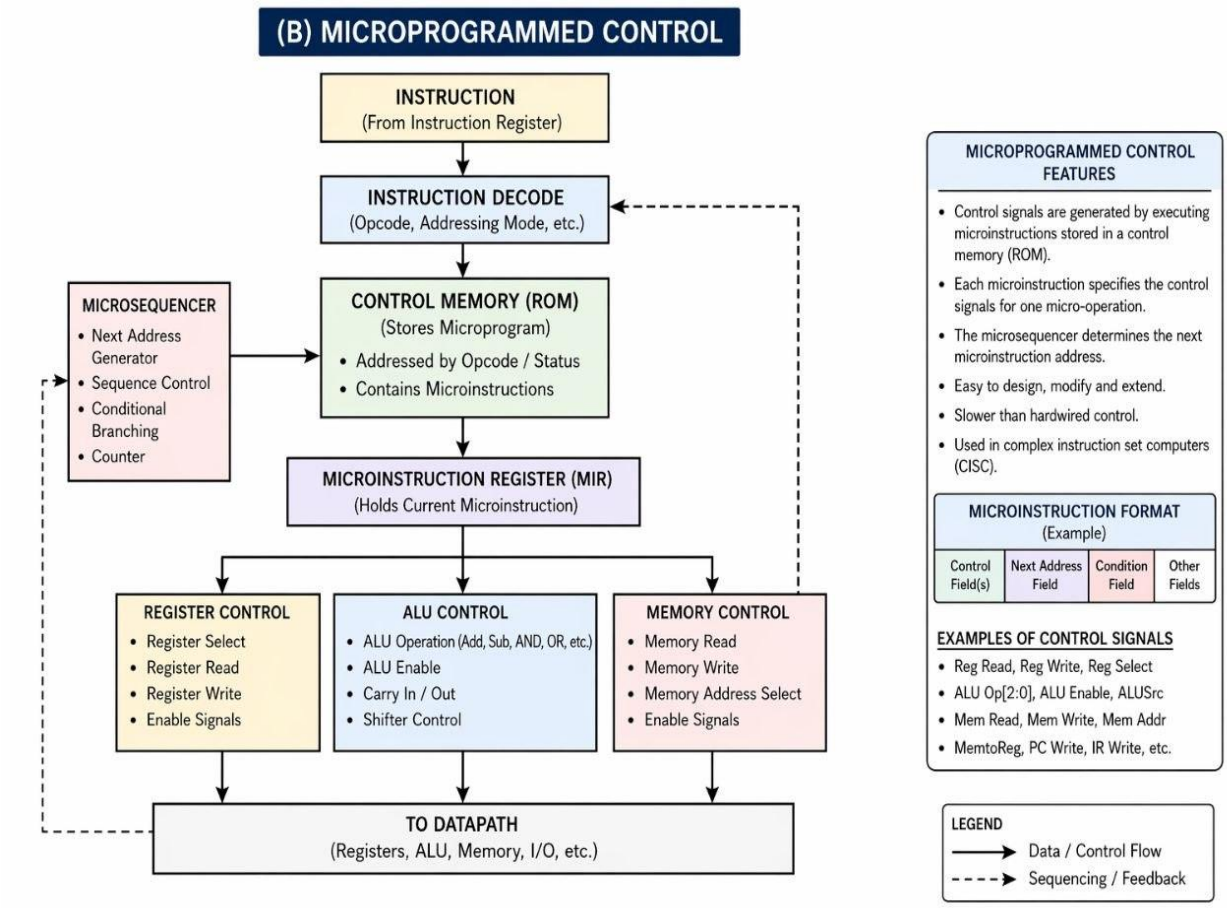


2.2 Microprogrammed Control

Microprogrammed control uses a "computer within a computer" approach. The control signals are stored in a special, high-speed memory called a **Control Store**.

- **How it works:** Each instruction is broken down into a series of **microinstructions**. These microinstructions reside in the Control Store (typically ROM). The control unit acts like a micro-sequencer that fetches these microinstructions to trigger the necessary hardware control signals.
- **Pros:**
    - **Highly Flexible:** You can change the instruction set or fix bugs simply by updating the microcode in the memory, without changing the hardware.
    - **Easier Design:** Managing a large, complex instruction set is much simpler when you are writing "code" rather than connecting thousands of logic gates.

- **Cons:**
    - **Slower:** Accessing the Control Store adds memory latency, making it slower than pure hardwired logic.
- **Best for:** Complex Instruction Set Computer (**CISC**) architectures (like older x86 processors) where instructions are very complex and varied.



➢ Summary Comparison

Feature	Hardwired Control	Microprogrammed Control
Implementation	Combinational circuits (gates)	ROM/Memory (Control Store)
Flexibility	Rigid (hard to change)	High (easy to reprogram)
Speed	Very Fast	Slower (Memory access overhead)
Design Complexity	High for complex instructions	Low (software-like approach)
Typical Use	RISC processors	CISC processors

3. Instruction Pipelining

Instruction pipelining is a technique used in modern processor design to increase instruction throughput—the number of instructions that can be executed in a unit of time—without necessarily reducing the execution time of a single instruction.

By overlapping the execution of multiple instructions, the processor keeps all its functional units busy, similar to an assembly line in a factory.

➢ The Fundamental Concept: The Assembly Line

In a non-pipelined processor, an instruction must complete its entire cycle before the next one begins. In a pipelined processor, the cycle is broken into distinct stages. As soon as one instruction completes a stage and moves to the next, the subsequent instruction enters the vacant stage.

➢ *Standard 5-Stage Pipeline*

Most classic RISC architectures utilize a five-stage pipeline:

1. **IF (Instruction Fetch):** The processor fetches the instruction from memory.
2. **ID (Instruction Decode):** The processor determines what the instruction needs to do and reads registers.
3. **EX (Execute):** The arithmetic logic unit (ALU) performs the operation.
4. **MEM (Memory Access):** If the instruction needs to read from or write to data memory, it happens here.
5. **WB (Write Back):** The result is written back to the register file.

➢ Key Advantages

- **Increased Throughput:** While the time to complete a *single* instruction (latency) remains the same or may even slightly increase due to pipeline overhead, the *total volume* of completed instructions increases significantly.
- **Efficiency:** It maximizes the utilization of hardware functional units that would otherwise sit idle while waiting for an instruction to finish its full cycle.

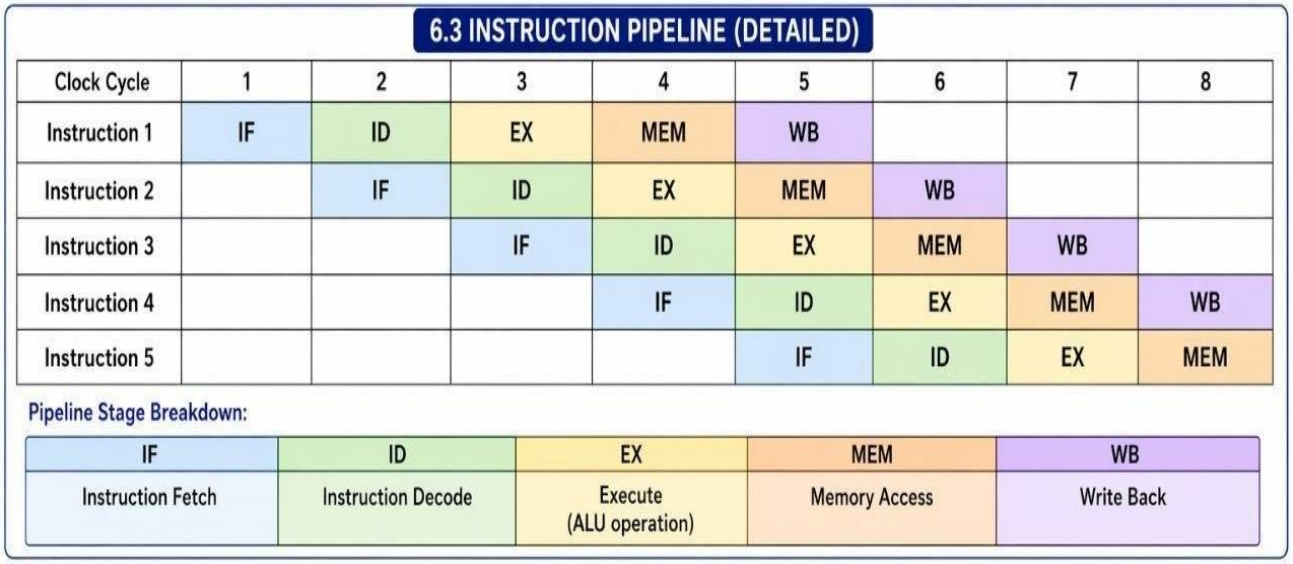
➢ Challenges: Pipeline Hazards

Pipelining is not without issues. Hazards occur when the next instruction cannot execute in the following clock cycle, causing the pipeline to "stall" (creating a "bubble").

- **Structural Hazards:** These occur when two or more instructions in the pipeline require the same hardware resource simultaneously (e.g., both trying to access memory).
- **Data Hazards:** These happen when an instruction depends on the result of a previous instruction that is still in the pipeline and hasn't written its result back yet.
- **Control Hazards (Branch Hazards):** These occur when the pipeline makes a decision based on the result of a branch instruction (like an if statement) that has not yet been resolved. If the branch is taken, instructions already in the pipeline must be flushed.

➢ Solutions to Hazards

- **Forwarding (Bypassing):** Passing the result of an operation directly to a subsequent instruction before it is formally written back to the register.
- **Pipeline Flushing:** Discarding instructions in the pipeline when a branch misprediction occurs.
- **Branch Prediction:** Using hardware logic to guess the outcome of a branch before it is executed, allowing the pipeline to continue pre-fetching instructions.



6.3 INSTRUCTION PIPELINE (DETAILED)

Clock Cycle	1	2	3	4	5	6	7	8
Instruction 1	IF	ID	EX	MEM	WB			
Instruction 2		IF	ID	EX	MEM	WB		
Instruction 3			IF	ID	EX	MEM	WB	
Instruction 4				IF	ID	EX	MEM	WB
Instruction 5					IF	ID	EX	MEM

Pipeline Stage Breakdown:

IF	ID	EX	MEM	WB
Instruction Fetch	Instruction Decode	Execute (ALU operation)	Memory Access	Write Back

4. Instruction-Level Parallelism (ILP)

Instruction-Level Parallelism (ILP) : is a measure of how many operations in a computer program can be performed simultaneously. While pipelining focuses on overlapping different stages of different instructions, ILP aims to execute multiple instructions in parallel during the *same* clock cycle.

In essence, if two instructions do not depend on each other, they can be processed at the same time to speed up the overall execution.

➢ Core Approaches to Achieving ILP

There are two primary ways to implement ILP: through hardware (dynamic) or through the compiler (static).

1. Hardware-Based (Dynamic ILP)

The processor hardware decides at runtime which instructions to execute in parallel.

- **Superscalar Execution:** The CPU has multiple execution units (ALUs, FPUs). It fetches multiple instructions and dispatches them to different units simultaneously.

- **Out-of-Order (OoO) Execution:** The processor executes instructions as soon as their inputs are available, rather than in the exact order they appear in the program.
- **Register Renaming:** A technique used to eliminate "false" data dependencies (like writing to the same register name) by mapping architectural registers to a larger pool of physical registers.

2. Software-Based (Static ILP)

The compiler analyzes the code and bundles independent instructions together before the program even runs.

- **VLIW (Very Long Instruction Word):** The compiler packs multiple independent operations into a single, very large instruction word. The hardware then executes all parts of that word in parallel without needing to check for dependencies.
- **Loop Unrolling:** The compiler replicates the body of a loop multiple times to reduce overhead and increase the number of instructions available for parallel execution.

➢ Barriers to ILP (Dependencies)

The amount of parallelism possible is strictly limited by dependencies between instructions.

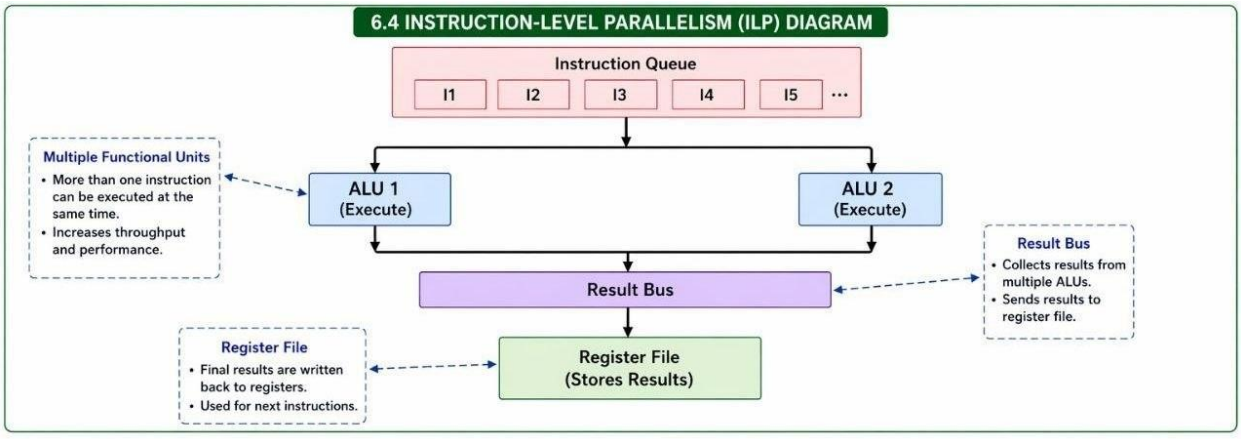
Dependency Type	Description	Resulting Hazard
Data Dependency	An instruction needs the result of a previous one.	**RAW** (Read After Write)
Name Dependency	Two instructions use the same register name but don't share data.	**WAR** (Write After Read) or **WAW**
Control Dependency	An instruction's execution depends on a branch (e.g., if statement).	Branch Misprediction Stalls

➢ ILP vs. Pipelining

It is helpful to think of the difference in terms of a factory:

- **Pipelining** is like an assembly line where one car starts as the previous one moves to the next station.
- **ILP** is like having *multiple* assembly lines running side-by-side in the same factory.

Most modern high-performance processors combine both: they are **pipelined superscalar** processors, meaning they have multiple pipelines working in parallel.



➢ Why ILP is Important ?

Instruction-Level Parallelism (ILP) is the engine behind modern computing performance. It is the primary reason why your programs run significantly faster today than they did decades ago, even when raw clock speeds (GHz) have hit a plateau.

Here is why ILP is critically important for computing architecture and software development:

1. The End of "Free" Speed (Power Wall)

Historically, engineers increased CPU performance simply by increasing the clock frequency. However, increasing clock speed increases power consumption and heat output exponentially ($Power \approx Frequency^3$).

- **The Importance of ILP:** Because we cannot keep turning up the clock speed without melting the chip, we must improve **Instruction Throughput**. ILP allows processors to do more work in the same amount of time at a lower, more manageable clock speed.

2. Maximizing Hardware Utilization

In a non-ILP processor, if you have a complex task, the processor's functional units (like the ALU for math or the FPU for floating-point calculations) often sit idle waiting for the next instruction.

- **The Importance of ILP:** ILP treats the processor like a busy kitchen. Instead of one chef doing one task at a time, ILP allows multiple "chefs" (execution units) to work on different ingredients (instructions) simultaneously. This ensures the expensive hardware silicon is constantly being utilized, providing a better return on investment for the chip design.

3. Scaling Performance for Complex Software

Modern applications—especially those involving heavy data processing, graphics rendering, or database management (like the projects you are working on)—are composed of millions of instructions.

The Importance of ILP: Even if a single instruction takes a fixed amount of time to execute, ILP allows the processor to "hide" this latency by working on subsequent, independent instructions in the background. Without ILP, software performance would be bottlenecked by the sequential speed of a single pipeline, making modern user interfaces and complex applications feel sluggish and unresponsive.

➢ Summary of Impact

Factor	Without ILP	With ILP
Performance	Limited to 1 instruction per cycle.	Multiple instructions per cycle.
Efficiency	Hardware units are frequently idle.	Hardware units remain active.
Scalability	Tied directly to clock speed increases.	Scales with architectural complexity.

Conclusion

The evolution of computer architecture demonstrates a persistent shift from simple, sequential processing toward complex, parallel execution. As explored, the realization of the **Control Unit**—whether through the blistering speed of **Hardwired Control** or the versatile flexibility of **Microprogrammed Control**—sets the foundation for how a processor interprets its instructions. However, the true leap in modern efficiency is found in the management of the datapath through **Pipelining** and **Instruction-Level Parallelism (ILP)**.

Pipelining allows a processor to handle multiple instructions in various stages of completion simultaneously. While this introduces challenges known as "hazards"—structural, data, and control conflicts—architectural innovations like forwarding and branch prediction have largely mitigated these bottlenecks. Furthermore, ILP takes this a step further by utilizing multiple execution units (Superscalar) and Out-of-Order execution to maximize hardware utilization.

In conclusion, the goal of computer architecture remains the same: to provide the most computation for the least cost and power. As raw clock frequencies have plateaued due to thermal constraints, the focus has shifted entirely toward architectural ingenuity. By refining the synergy between the ISA, the datapath, and parallel execution strategies, designers continue to scale performance. Understanding these functional units is not just an exercise in hardware design, but a necessity for anyone looking to optimize software and understand the fundamental limits of modern computing.

References

1. Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann. (The definitive text on ILP, pipelining, and power-efficient design).
2. Stallings, W. (2022). Computer Organization and Architecture: Designing for Performance (11th ed.). Pearson. (Excellent resource for Control Unit realization and Bus organization).
3. Hamacher, V. C., Vranesic, Z. G., & Zaky, S. G. (2011). Computer Organization and Embedded Systems. McGraw-Hill. (Focuses on the functional units and hardware implementation steps).
4. Tanenbaum, A. S., & Austin, T. (2012). Structured Computer Organization. Prentice Hall. (Provides a clear breakdown of the multi-level view of computers, from ISA to logic gates).
5. Null, L., & Lobur, J. (2018). The Essentials of Computer Organization and Architecture. Jones & Bartlett Learning. (Covers the basics of datapaths, MAR/MDR registers, and bus types).